

1 Consigne d'examen – S1

Examen – Mini-application de réservation de documents au CDM

Classe : S1 – Durée : 2h maximum

Tu vas réaliser une **petite application web complète** qui simule une nouvelle fonctionnalité de la plateforme de l'école : la **réservation de documents** (livres, DVD, CD, etc.) au **Centre de Documentation Multimédia (CDM)**.

🎯 L'objectif : permettre à un élève de **faire une demande d'emprunt** de plusieurs documents, indiquer la **durée de l'emprunt**, puis voir le **statut de sa demande**.

Ton application doit comporter :

- un **back-end** en Python (**FastAPI**),
- 🗄️ une **base de données SQLite**,
- 🎨 une **interface front-end simple** en **React + TypeScript (TSX)** (ou au minimum des tests via *Swagger* `/docs`).

1 Scénario fonctionnel

Un élève veut emprunter des documents au CDM pour une certaine durée.

Ton application doit permettre :

🔗 1. Remplir un formulaire d'accès / demande

L'élève doit saisir :

- **nom**
- **prénom**
- 📷 **classe**
- 🎂 **âge**
- 📅 **date de la demande**

Ces informations serviront à créer la **demande d'emprunt**.

🔗 2. Afficher la liste des documents disponibles au CDM

- Au moins **10 documents** enregistrés dans la base (mélange possible de livres, DVD, CD).
- Chaque document dans la base doit avoir :
 - 📄 **id**
 - 🖼️ **titre**
 - **type** ("LIVRE", "DVD", "CD", ...)

L'élève doit pouvoir **voir cette liste** (via Swagger ou via une petite interface front-end).

3. Choisir le nombre d'articles à emprunter

- L'élève indique **combien d'articles** il veut emprunter (par exemple : , ou).
- Pour chaque article (article 1, article 2, article 3, ...):
- il **sélectionne un document** dans la liste (par son ou son).

Tu peux représenter cette sélection :

- soit comme une **liste d'identifiants (ids)** de documents,
 - soit comme un simple **texte récapitulatif**.
-

4. Indiquer la durée de l'emprunt

- L'élève doit préciser pour **combien de jours** il souhaite emprunter les documents (ex : ,).
 - Cette durée doit être **enregistrée** dans la demande.
-

5. Enregistrer la demande d'emprunt

La demande est enregistrée dans la base de données avec un **statut** :

- au moment de l'enregistrement : .

La demande contient au minimum :

- les **informations de l'élève** (nom, prénom, classe, âge, date de demande),
 - la **liste des documents demandés** (ou un résumé),
 - la **durée d'emprunt**,
 - le **statut** (, puis éventuellement).
-

6. Confirmer au front / à l'utilisateur

Après enregistrement, ton API renvoie un **message de confirmation**, par exemple :

« Demande d'emprunt enregistrée. Statut actuel : EN_COURS. Merci de patienter que la responsable du CDM l'approuve. »

Ce message doit être **visible** soit dans Swagger () , soit dans ton **front-end React**.

✓ 7. Consultation des demandes et approbation – *Bonus conseillé*

Tu peux ajouter :

- une route pour **lister toutes les demandes** :
- `GET /demandes` → renvoie la liste des demandes enregistrées.
- une route pour **changer le statut** d'une demande (par exemple par la responsable du CDM) :
- `"EN_COURS" → "APPROUVÉE"`.

Message possible après approbation :

« Demande approuvée. Vous pouvez passer au CDM pour récupérer vos documents. »

2 Partie Back-end (FastAPI)

Tu dois implémenter au minimum les **endpoints** suivants :

 **GET** `/documents`

Renvoie la **liste des documents disponibles** (au format JSON) :

- `id`
- `titre`
- `type`

 **POST** `/demandes`

Crée une **nouvelle demande d'emprunt** à partir de :

- les **informations de l'élève**,
- la **liste des documents demandés** (ou une liste d'ids),
- la **durée d'emprunt**,
- définit `statut = "EN_COURS"`,
- enregistre la demande en base,
- renvoie un **message de confirmation**.

 **GET** `/demandes`

Renvoie la **liste des demandes enregistrées** (utile pour vérifier pendant l'examen).

+ Bonus (facultatif mais recommandé)

 **PATCH** /demandes/{id}/approuver

Change le **statut** d'une demande :

• de "EN_COURS" → "APPROUVÉE".

Tu peux tester toutes ces routes dans **Swagger** (/docs).

3 Partie Base de données (SQLite)

Tu utiliseras une base **SQLite** (par exemple `cdm.db`) avec au minimum :

♥ **Table** documents

-  `id` (clé primaire)
-  `titre`
- `type` (LIVRE, DVD, CD, ...)

♥ **Table** demandes

-  `id`
- `nom`
- `prenom`
-  `classe`
-  `age`
-  `date_demande`
- `duree_jours`
- `statut` (EN_COURS, APPROUVÉE)
-  (facultatif) `details_documents` (texte contenant un **résumé des documents demandés**)

Pour **simplifier l'examen**, la liste des documents empruntés peut être stockée dans un **champ texte** (`details_documents`) plutôt que dans une deuxième table.

4 Partie Front-end (React + TypeScript – TSX)

(si tu fais une petite interface en plus de Swagger)

Tu peux créer / compléter une interface simple dans `App.tsx` qui permet :

🔗 Formulaire de demande

Champs :

- `nom`
- `prénom`
-  `classe`

- 🎂 âge
 - 🎺 date de demande
 - durée d'emprunt (en jours)
-

📄 Affichage des documents

- Bouton « **Charger les documents** » → appelle `GET /documents` et affiche la liste (titre + type).
 - L'élève peut indiquer quels documents il veut emprunter (par exemple en saisissant leurs `id` ou en sélectionnant dans une liste).
-

📤 Envoi de la demande

- Bouton « **Envoyer la demande** » → appelle `POST /demandes` avec :
 - les **infos de l'élève**,
 - la **durée**,
 - la **liste des documents** souhaités.
 - Le front doit **afficher le message de confirmation** renvoyé par l'API.
-

+ Bonus

- Ajouter un bouton « **Voir les demandes** » → appelle `GET /demandes` et affiche un **tableau simple** avec les demandes (nom, classe, durée, statut).
-

5 Prompt engineering (fortement recommandé)

Tu peux utiliser une **IA** pour t'aider , mais :

- tu dois rédiger des **prompts clairs et structurés**, par exemple :
- pour **concevoir les modèles FastAPI** (modèles Pydantic) + la **structure des tables** SQLite,
- pour t'aider à écrire les **endpoints** (`GET /documents` , `POST /demandes` , `GET /demandes`) ou une petite interface React.
- garde tes **prompts** (et éventuellement un **résumé des réponses**) :
 - en **commentaire** dans ton code, ou
 - dans un **fichier texte** séparé.

Ces prompts font partie de ton travail : ils montrent ta capacité à bien communiquer avec une IA .

6 Évaluation 🧐

Ta note prendra en compte :

- le **bon fonctionnement** des endpoints (/documents , /demandes),
- la **présence** et la **cohérence** de la base de données (documents , demandes),
- la capacité de ton **appli** (Swagger ou front) à :
- afficher la **liste des documents**,
- **créer une demande**,
- **afficher un message de confirmation**,
- la **qualité de ton code** (lisible, structuré, logique),
- la **qualité de tes prompts** (clairs, précis, pertinents).

💖 Bonus

Des points supplémentaires pourront être attribués si tu :

- gères le **statut** APPROUVÉE (changement après validation),
- affiches la **liste des demandes** dans une interface,
- ajoutes des **validations simples** (ex : âge positif, durée d'emprunt > 0, etc.),
- soignes particulièrement la **présentation** (nom des champs, messages clairs).